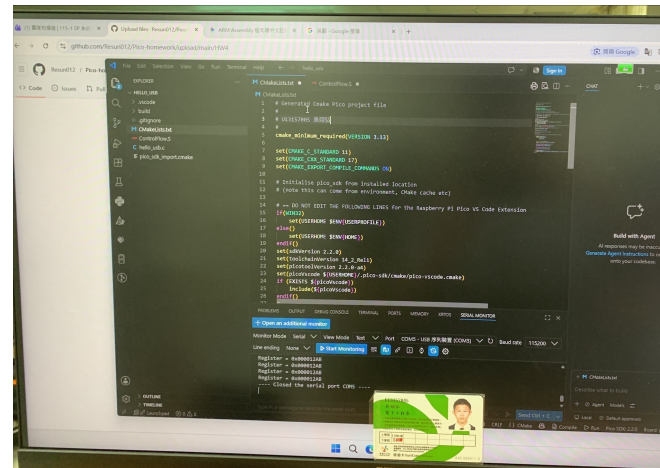


ControlFlow.S

```

1  @ =====
2  @ 該文件包含了第 5 章中的各種程式碼片段。
3  @ 這樣既能確保它們能夠編譯通過，
4  @ 也方便你對其進行單步調試。
5  @ 這些程式碼片段都已標記，
6  @ 因此你可以在感興趣的程式碼處設定斷點。
7  @ 程式碼片段之後是用於
8  @ 將暫存器內容轉換為 ASCII 碼的例程。
9  @
10 @ U13157005 吳翊弘
11 @
12 @ =====
13
14 .thumb_func          @ 必須宣告，因為 SDK 的呼叫指令（BLX）需要使用 Thumb 模式
15 .global main          @ 向連結器（Linker）提供程式進入點
16
17 main:    BL    stdio_init_all @ 初始化 UART 或 USB 序列埠輸出
18
19 @ 若想觀察無窮迴圈，請取消下方兩行的註解：
20 11: @ MOV R1, #1
21     @ B 11
22
23 12: CMP R4, #45
24     BEQ 11          @ 若 R4 == 45，則跳躍至 11
25
26 13: MOV R2, #1        @ R2 用作計數器 I (I = 1)
27 loop: @ --- 迴圈主體區塊 ---
28     @ 大部分的判斷邏輯放置在迴圈結尾（Do-While 結構）
29     ADD R2, #1        @ I = I + 1
30     CMP R2, #10
31     BLE loop          @ 若 I <= 10，繼續執行 loop
32
33 14: MOV R2, #10        @ R2 用作計數器 I (I = 10)
34 loop2: @ --- 倒數迴圈主體區塊 ---
35     @ 此處不需要額外的 CMP 指令，因為 SUBS/SUB 會自動更新條件旗標（Z flag）
36     SUB R2, #1        @ I = I - 1
37     BNE loop2         @ 若 I != 0 (Z flag 未被置位)，繼續執行 loop2
38
39 15: @ 假設 R4 代表變數 X，在此進行初始化
40     MOV R4, #5
41 loop3: CMP R4, #5
42     BGE loopdone      @ 若 R4 >= 5，跳出迴圈至 loopdone (While 條件判斷)
43     @ ... 此處可放置其他迴圈主體敘述 ...
44     MOV R4, #6
45     B loop3
46 loopdone: @ 迴圈結束後繼續執行此處程式
47
48 16: CMP R5, #10
49     BGE elseclause    @ 若 R5 >= 10，跳躍至 else 區塊
50     @ ... If 區塊內容 (R5 < 10 時執行) ...
51     B endif

```



```

52  elseclause:
53      @ ... Else 區塊內容 (R5 >= 10 時執行) ...
54  endif: @ If-Else 條件判斷結束後的繼續執行點
55
56  17: @ --- 位元遮罩 (Masking) 操作：保留最高有效位元組 (MSB) ---
57      MOV R5, #0xFF
58      LSL R5, #24      @ 將 0xFF 左移 24 位元，R5 變為 0xFF000000
59      AND R6, R5      @ R6 與 R5 進行及閘 (AND) 操作，只保留 R6 的最高位元組
60
61  18: @ --- 位元強制設定 (Set Bits) 操作 ---
62      MOV R5, #0xFF
63      ORR R6, R5      @ R6 與 R5 進行或閘 (OR) 操作，將 R6 的最低位元組強制設為 1 (0xFF)
64
65  19: @ --- 位元清除 (Bit Clear) 操作 ---
66      BIC R6, R5      @ R6 = R6 AND NOT(R5)，清除 R6 中對應 R5 為 1 的位元 (清除最低位元組)
67
68  @ =====
69  @ 範例：將暫存器 (Register) 內的數值轉換為 ASCII 十六進位字串
70  @
71  @ 暫存器配置說明：
72  @ R0, R1 - 傳遞給 printf 的參數
73  @ R1      - 同時用作寫入字元記憶體位址的指標
74  @ R4      - 欲印出數值的暫存器
75  @ R5      - 迴圈計數器 (Loop Index)
76  @ R6      - 目前處理中的字元/十六進位數字
77  @ R7      - 暫存用的臨時暫存器
78  @ =====
79
80  printexample:
81      @ 將 0x12AB 載入至 R4 中
82      MOV R4, #0x12    @ 設定高位元組
83      LSL R4, #8        @ R4 = 0x1200
84      MOV R7, #0xAB     @ 設定低位元組
85      ADD R4, R7        @ R4 = 0x12AB (欲轉換的數值)
86
87      LDR R1, =hexstr @ 取得字串緩衝區的起始位址
88      ADD R1, #9        @ 將指標指向最右側 (最低有效位元) 的字元位置
89
90  @ 迴圈執行次數：FOR R5 = 8 DOWNT0 1
91      MOV R5, #8        @ 共計轉換 8 個十六進位數字 (32-bit)
92  loop4: MOV R6, R4
93      MOV R7, #0x0F     @ 建立遮罩以擷取最後 4 個位元 (1 個 Hex 數字)
94      AND R6, R7        @ 擷取最低有效數字 (Least Significant Digit)
95
96  @ 判斷數字為 0-9 還是 A-F
97      CMP R6, #10
98      BGE letter      @ 若 R6 >= 10，跳躍至字母處理區塊 (A-F)
99
100 @ 數字 0-9 的處理：轉為 ASCII 數字字元 ('0'-'9')
101     ADD R6, #'0'
102     B cont          @ 跳躍至 If-Else 結束處
103
104 letter: @ 數字 10-15 的處理：轉為 ASCII 字母字元 ('A'-'F')
105     ADD R6, #('A'-10)

```

```
106
107 cont:  @ --- If-Else 結束 ---
108     STRB R6, [R1] @ 將轉換後的 ASCII 字元寫入記憶體位址 [R1]
109     SUB  R1, #1    @ 將位址指標減 1，指向前一個（左側）字元位置
110     LSR  R4, #4    @ 將 R4 右移 4 位元，準備處理下一個數字
111
112     @ 更新迴圈計數器 R5
113     SUB  R5, #1    @ 遞減迴圈計數器
114     BNE  loop4     @ 若 R5 != 0，繼續執行迴圈
115
116 repeat:
117     LDR R0, =printstr @ 載入格式化字串 "%s\n"
118     LDR R1, =hexstr   @ 載入轉換完成的十六進位字串位址
119     BL  printf        @ 呼叫 printf 印出結果
120     B   repeat        @ 無窮迴圈重複印出
121
122 .align 4
123 .data
124 hexstr:  .asciz "0x12345678" @ 用於存放轉換結果的字串緩衝區
125 printstr: .asciz "Register = %s\n" @ printf 格式字串
```